

**Universidad de Antioquia**

Ingreniería de telecomunicaciones  
Informatica 2

## **Sweet Crush**

Proyecto de programación

**Estudiante:**

Andrés Felipe Sierra Fernández  
David Guardiola

**Docente:**

Anibal Jose Guerra

# 1 Contextualización

El desafío trata sobre un juego de tipo *arcade* conformado por diferentes fichas, en el cual el jugador podrá eliminar fichas del tablero, una por ronda, modificar la estructura del tablero, detectar combinaciones, entre otras acciones.

El juego exige que el tablero sea representado de una manera diferente a una matriz convencional de valores enteros. Las fichas deben estar representadas mediante una secuencia de bits empaquetados en bytes, haciendo que la forma de almacenar y manejar el tablero sea una parte importante del desafío.

Además, durante el juego el tablero puede cambiar su estructura, permitiendo agregar o eliminar filas y columnas, y se deben procesar las combinaciones que se produzcan después de las diferentes acciones realizadas. “

# 2 Análisis

El juego requiere que cada posición del tablero sea representada mediante 3 bits. Debido a que con 3 bits es posible representar 8 combinaciones diferentes, se utilizarán 6 de estas para representar las distintas fichas, una para representar una posición vacía y la última para representar una combinación especial.

Para implementar esta representación del tablero, es necesario desarrollar funciones que permitan administrar la memoria y realizar la manipulación de los bits correspondientes a cada ficha. Para esta primera parte del proyecto se utilizarán las siguientes funciones:

- `int calcularBytesTablero(int filas, int columnas)`: se encarga de calcular la cantidad de bytes necesarios para almacenar el tablero de acuerdo con sus dimensiones, teniendo en cuenta que cada posición requiere 3 bits.
- `unsigned char* crearTablero(int filas, int columnas, int bytes)`: se encarga de crear el tablero utilizando memoria dinámica, a partir de sus dimensiones y de la cantidad de bytes calculada.
- `void inicializarBuffer(unsigned char* buffer, int bytesTablero)`: recibe un puntero a la memoria del tablero y la cantidad de bytes que este utiliza, para inicializar la memoria con ceros. Estos valores serán modificados posteriormente a medida que se almacenen las fichas.
- `int calcularPosicion(int fila, int columna, int columnas)`: calcula la posición lineal correspondiente a una determinada fila y columna del tablero, permitiendo posteriormente acceder a la ficha almacenada en dicha posición.
- `void guardarFicha(unsigned char* memoria, int posicion, unsigned char ficha)`: recibe un puntero a la memoria del tablero, la posición de la ficha y el valor correspondiente a esta. Su función es almacenar los 3 bits de la ficha en la posición correspondiente dentro de la secuencia de bits.

- `unsigned char obtenerFicha(unsigned char* memoria, int posicion):`  
se encarga de extraer los 3 bits correspondientes a una ficha a partir de su posición dentro de la memoria, permitiendo recuperar el valor almacenado.

Estas funciones constituyen la base para la implementación del tablero y permiten abstraer las operaciones relacionadas con la administración de la memoria y la manipulación de los bits. A medida que avance el desarrollo del programa, se implementarán nuevas funciones encargadas de las demás operaciones requeridas por el juego, como la detección de combinaciones, eliminación de fichas y modificación de la estructura del tablero. Una vez establecida esta base, se podrá continuar con la implementación de la parte visual del juego. “ “ “ latex

### 3 Diseño

Para la representación del tablero se decidió almacenar las fichas utilizando grupos de 3 bits dentro de una secuencia de bytes. Los bits correspondientes a cada ficha se van almacenando de izquierda a derecha dentro de cada byte, teniendo en cuenta que una ficha puede atravesar el límite entre dos bytes consecutivos.

Un ejemplo de la representación del tablero en memoria es el siguiente:

```
Byte[0]: 00101001
Byte[1]: 11001011
Byte[2]: 01100011
Byte[3]: 01000100
Byte[4]: 00010001
Byte[5]: 00101010
Byte[6]: 01110010
Byte[7]: 10100000
```

Bytes del tablero: 8

A partir de esta representación, la función `obtenerFicha` permite extraer el valor correspondiente a cada posición del tablero.

| Posición | Valor de ficha | Representación binaria |
|----------|----------------|------------------------|
| 0        | 1              | 001                    |
| 1        | 2              | 010                    |
| 2        | 3              | 011                    |
| 3        | 4              | 100                    |
| 4        | 5              | 101                    |
| 5        | 5              | 101                    |
| 6        | 4              | 100                    |
| 7        | 3              | 011                    |

Para realizar esta operación se debe tener en cuenta la posición de los 3 bits dentro de la memoria, ya que algunas fichas pueden encontrarse completamente

dentro de un byte, mientras que otras pueden estar divididas entre dos bytes consecutivos. En este último caso, la ficha puede tener dos bits almacenados en un byte y el bit restante en el siguiente, o un bit en un byte y los otros dos en el siguiente.

Una vez obtenidos los valores de las fichas, se utiliza un ciclo para recorrer las posiciones del tablero y representarlo en consola. De esta manera, la información almacenada linealmente en memoria puede ser mostrada al usuario como filas y columnas:

```
fila 0: B C D E
fila 1: F F E D
fila 2: C B A B
fila 3: A E F C
fila 4: D E F C
```

En el estado actual del desarrollo, el programa permite calcular la cantidad de bytes necesarios para el tablero, crear la memoria dinámica correspondiente, calcular la posición lineal de cada ficha, almacenar y obtener sus valores y finalmente mostrar el tablero en pantalla.

La siguiente etapa del desarrollo consiste en implementar la redimensión del tablero. Para esto será necesario calcular nuevamente la cantidad de bytes requerida, crear un nuevo bloque de memoria, reorganizar las fichas de acuerdo con las nuevas dimensiones, liberar la memoria utilizada por el tablero anterior y finalmente mostrar la nueva estructura del tablero. “

““latex

### 3.1 Generación y llenado del tablero

Una vez implementadas las funciones necesarias para almacenar y recuperar fichas mediante la representación de tres bits, se desarrollaron las funciones encargadas de generar las fichas y llenar el tablero.

La función `generarFicha()` se encarga de generar aleatoriamente el tipo de ficha que será almacenado. Debido a que el juego cuenta con seis tipos diferentes de fichas, se utiliza la función `rand()` junto con el operador módulo para obtener un valor en el rango de 0 a 5.

```
unsigned char generarFicha(){
    return rand() % 6;
}
```

Por otra parte, la función `llenarTablero()` recorre todas las posiciones del tablero mediante las filas y columnas. Para cada posición recorrida se genera una ficha aleatoria y posteriormente se calcula su posición lineal dentro de la secuencia de fichas. Finalmente, la ficha generada es almacenada en dicha posición mediante la función `guardarFicha()`.

```

void llenarTablero(unsigned char *tablero, int filas, int columnas){
    for(int fila = 0; fila < filas; fila++){
        for(int columna = 0; columna < columnas; columna++){
            unsigned char ficha = generarFicha();
            int posicion = calcularPosicion(fila, columna, columnas);
            guardarFicha(tablero, posicion, ficha);
        }
    }
}

```

De esta manera, las posiciones del tablero son recorridas de forma ordenada, mientras que el valor de la ficha asignada a cada posición se genera aleatoriamente. La función no modifica el orden de las posiciones ni realiza una distribución aleatoria de estas, sino que genera un valor diferente para cada posición durante el recorrido. “ “ $\text{latex}$  “ $\text{latex}$

### 3.2 Búsqueda y registro de combinaciones

Después de construir las funciones encargadas de revisar las combinaciones horizontales y verticales, se modificó la función `buscarCombinaciones()` para que, además de identificar las posiciones que deben eliminarse, permita conocer cuántas combinaciones fueron detectadas en el tablero.

La función recorre todas las posiciones del tablero y utiliza `combHorizontales()` y `combVerticales()` para determinar cuántas fichas iguales consecutivas existen a partir de cada posición.

Para evitar contar varias veces una misma combinación, se verifica si la posición actual corresponde al inicio de la secuencia. En el caso horizontal, se compara la ficha actual con la ficha ubicada en la columna anterior. Si son iguales, significa que la combinación ya había comenzado anteriormente y, por lo tanto, no se vuelve a contar.

```

bool inicioHorizontal = true;

if(columna > 0){
    int posicionAnterior =
        calcularPosicion(fila, columna - 1, columnas);

    unsigned char fichaAnterior =
        obtenerFicha(tablero, posicionAnterior);

    if(fichaAnterior == ficha){
        inicioHorizontal = false;
    }
}

```

El mismo procedimiento se realiza para las combinaciones verticales, comparando la ficha actual con la que se encuentra en la fila anterior. De esta

manera, una combinación de cuatro o cinco fichas se cuenta como una sola combinación y no como varias combinaciones consecutivas.

Cuando se encuentra una combinación válida, es decir, de tres o más fichas iguales, se incrementa el contador de combinaciones y se recorren las posiciones que forman parte de ella. Cada posición se agrega al arreglo `eliminadas`, siempre verificando primero mediante `estaEliminada()` que no haya sido registrada anteriormente.

Finalmente, la cantidad de combinaciones encontradas se acumula en la variable `combinacionDetectadas`, mientras que la función retorna la cantidad total de posiciones que fueron registradas para su eliminación.

De esta forma, la función permite obtener dos resultados diferentes del proceso: por un lado, las posiciones que deben eliminarse y, por otro, la cantidad de combinaciones que fueron encontradas en el tablero. “

La función `combVerticales()` utiliza la misma lógica, pero en este caso el recorrido se realiza hacia abajo. Se toma la ficha de la posición inicial como referencia y se comparan las fichas de las filas siguientes en la misma columna. El contador aumenta mientras las fichas sean iguales y el recorrido termina cuando aparece una ficha diferente o se llega al final del tablero.

```
int combVerticales(unsigned char* tablero, int fila, int columna,
                  int filas, int columnas){
    if (tablero == 0) return -1;
    if (fila < 0 || columna < 0 || filas <= 0 || columnas <= 0) return -1;
    if (fila >= filas || columna >= columnas) return -1;

    int posicion = calcularPosicion(fila, columna, columnas);
    unsigned char fichaBase = obtenerFicha(tablero, posicion);

    int contador = 1;
    int f = fila + 1;

    while (f < filas) {
        posicion = calcularPosicion(f, columna, columnas);
        unsigned char ficha = obtenerFicha(tablero, posicion);

        if (ficha != fichaBase) {
            break;
        }

        contador++;
        f++;
    }

    return contador;
}
```

Con estas dos funciones se puede conocer cuántas fichas iguales consecutivas

existen a partir de una posición determinada, tanto en dirección horizontal como vertical. Esto permite utilizar posteriormente esta información para determinar si existe una combinación válida y realizar las acciones correspondientes sobre el tablero. “ “`latex`

### 3.3 Búsqueda y registro de combinaciones

Después de construir las funciones encargadas de revisar las combinaciones horizontales y verticales, se creó la función `buscarCombinaciones()`, cuya función es recorrer todo el tablero y utilizar estas funciones para encontrar las combinaciones de tres o más fichas iguales.

Para cada posición del tablero se revisan las dos direcciones. Si se encuentra una combinación horizontal o vertical de al menos tres fichas, se calculan las posiciones que forman parte de ella y se almacenan en el arreglo `eliminadas`. Antes de agregar una posición se utiliza la función `estaEliminada()` para comprobar si esta ya había sido registrada. Esto es necesario porque una misma ficha puede formar parte de una combinación horizontal y de una vertical al mismo tiempo.

La función `estaEliminada()` realiza una búsqueda sencilla dentro del arreglo de posiciones ya registradas. Si encuentra la posición que se está consultando, retorna `true`; de lo contrario, retorna `false`.

La función `buscarCombinaciones()` finalmente retorna la cantidad de posiciones que fueron registradas para su posterior eliminación. De esta manera, el arreglo `eliminadas` funciona como una lista de las posiciones que deben ser procesadas posteriormente, sin modificar todavía el contenido del tablero.

Un fragmento importante de esta lógica es la comprobación de posiciones repetidas:

```
if(!estaEliminada(eliminadas, cantidadEliminadas, posicion)){
    eliminadas[cantidadEliminadas] = posicion;
    cantidadEliminadas++;
}
```

Con esto se evita que una misma posición sea almacenada varias veces cuando pertenece a más de una combinación. “ “

“`latex`

### 3.4 Eliminación, caída y rellenado del tablero

Una vez identificadas las posiciones que forman parte de una combinación, se desarrollaron las funciones necesarias para modificar el tablero y permitir que las fichas ocupen nuevamente los espacios disponibles.

La función `eliminarFichas()` recibe el arreglo de posiciones encontradas por `buscarCombinaciones()` y modifica cada una de ellas utilizando el valor 6. Este valor se utiliza para representar una posición vacía dentro del tablero.

```

void eliminarFichas(unsigned char* tablero, int* eliminadas,
                   int cantidadEliminadas){
    for(int i = 0; i < cantidadEliminadas; i++){
        guardarFicha(tablero, eliminadas[i], 6);
    }
}

```

Después de eliminar las fichas, es necesario reorganizar las posiciones restantes. Para esto se creó la función `hacerCaerTablero()`, encargada de recorrer cada una de las columnas y llamar a `hacerCaerColumna()` para realizar el proceso de caída.

```

void hacerCaerTablero(unsigned char* tablero, int filas, int columnas){
    if(tablero == 0) return;
    if(filas <= 0 || columnas <= 0) return;

    for(int columna = 0; columna < columnas; columna++){
        hacerCaerColumna(tablero, columna, filas, columnas);
    }
}

```

Finalmente, se implementó `rellenarTablero()`, cuya función es recorrer nuevamente el tablero y buscar las posiciones que todavía se encuentran vacías. Cada posición vacía es reemplazada por una nueva ficha generada aleatoriamente mediante `generarFicha()`.

```

if(ficha == 6){
    unsigned char nuevaFicha = generarFicha();
    guardarFicha(tablero, posicion, nuevaFicha);
}

```

De esta forma, después de encontrar y eliminar una combinación, el tablero puede reorganizarse y volver a llenarse para continuar con el juego. Este proceso también permite que las nuevas posiciones generadas produzcan otras combinaciones, las cuales pueden ser detectadas posteriormente mediante una nueva búsqueda. “ “ “ “ latex

### 3.5 Procesamiento de eliminaciones y modificación del tablero

Además de las eliminaciones producidas por las combinaciones, se implementó una función para procesar la eliminación de una ficha seleccionada directamente por el jugador. La función `procesarEliminacionFicha()` solicita una posición válida dentro del tablero y, si la posición es correcta, reemplaza la ficha correspondiente por el valor 6, que representa una posición vacía.

Después de realizar la eliminación, se ejecutan las funciones encargadas de hacer caer las fichas y rellenar los espacios disponibles. Finalmente, se muestra nuevamente el tablero para observar el resultado de la acción.

```

guardarFicha(tablero, posicion, 6);

hacerCaerTablero(tablero, filas, columnas);

rellenarTablero(tablero, filas, columnas);

mostrarTablero(tablero, filas, columnas);

```

También se implementó la función `agregarFila()`, con la cual se recibe la posición donde será agregada la ficha. Para realizar este proceso se calcula primero el tamaño necesario para almacenar el tablero original y el tamaño que tendrá después de agregar la nueva fila.

Debido a que el tablero utiliza memoria dinámica, se crea un nuevo espacio de memoria con el tamaño correspondiente a las nuevas dimensiones. Posteriormente, se recorre el nuevo tablero y se copian las fichas del tablero anterior, dejando la posición indicada para la nueva fila. Las posiciones de esta fila se llenan utilizando fichas generadas aleatoriamente.

Una vez terminada la copia, se libera la memoria utilizada por el tablero anterior y el puntero pasa a utilizar el nuevo espacio de memoria. Finalmente, se actualiza la cantidad de filas del tablero.

```

delete[] tablero;

tablero = nuevoTablero;
filas = nuevasFilas;

```

Con esto, la modificación del tablero no se limita al contenido de sus fichas, sino que también permite cambiar dinámicamente su estructura. El mismo principio puede utilizarse posteriormente para implementar la eliminación de filas y las modificaciones relacionadas con las columnas. “ “`“latex id=“9h4m2k”`”

### 3.6 Eliminación dinámica de filas

Para complementar la posibilidad de agregar filas, se implementó la función `eliminarFila()`, que permite reducir el tamaño del tablero eliminando una fila determinada.

Primero se realizan las validaciones necesarias para comprobar que el tablero sea válido y que todavía sea posible eliminar una fila. Luego se calcula el nuevo número de filas y la cantidad de bytes que se necesitan para almacenar el tablero con su nuevo tamaño.

Al igual que en `agregarFila()`, se crea un nuevo bloque de memoria para almacenar el tablero. En este caso, se recorren las filas del nuevo tablero y se copian las fichas del tablero anterior, omitiendo la fila que se indicó para eliminar. Para las filas que se encuentran después de la eliminada, se utiliza la posición correspondiente en el tablero anterior para mantener el orden de las fichas.

Una vez terminada la copia, se libera la memoria que utilizaba el tablero anterior y se actualiza el puntero para que apunte al nuevo tablero. Finalmente, se actualiza la cantidad de filas.

La parte principal del proceso de traslado de las fichas es:

```
int filaVieja = f;

if(f >= fila){
    filaVieja++;
}

for(int c = 0; c < columnas; c++){

    int posicionVieja = calcularPosicion(filaVieja, c, columnas);

    unsigned char ficha = obtenerFicha(tablero, posicionVieja);

    int posicionNueva = calcularPosicion(f, c, columnas);

    guardarFicha(nuevoTablero, posicionNueva, ficha);
}
```

De esta manera, la fila seleccionada queda fuera del nuevo tablero y las demás fichas se mantienen en el mismo orden. El proceso también permite mantener la representación del tablero mediante memoria dinámica, ajustando la cantidad de memoria utilizada a las nuevas dimensiones. “ “`latex`

### 3.7 Modificación dinámica de columnas

De forma similar a las filas, se implementaron las funciones `eliminarColumna()` y `agregarColumna()` para permitir modificar dinámicamente la cantidad de columnas del tablero.

La función `eliminarColumna()` recibe la columna que se desea eliminar y crea un nuevo tablero con una columna menos. Para construirlo, se recorren todas las posiciones del nuevo tablero y se copian las fichas correspondientes del tablero anterior, omitiendo la columna seleccionada.

Para determinar qué columna del tablero anterior corresponde a cada posición del nuevo tablero, se utiliza una condición que permite mantener las columnas anteriores a la eliminada y desplazar una posición las que se encuentran después de ella.

```
if(c < columna){
    columnaVieja = c;
}
else{
    columnaVieja = c + 1;
}
```

Una vez copiadas las fichas, se libera la memoria del tablero anterior, se asigna el nuevo bloque de memoria al puntero y se actualiza la cantidad de columnas.

Por otro lado, la función `agregarColumna()` realiza el proceso contrario. Se crea un nuevo tablero con una columna adicional y se recorren sus posiciones para copiar las fichas del tablero anterior. La nueva columna se genera en la posición indicada y sus fichas se obtienen mediante `generarFicha()`.

```
if(c == columna + 1){

    int posicionNueva =
        calcularPosicion(f, c, nuevasColumnas);

    guardarFicha(nuevoTablero, posicionNueva, generarFicha());
}
```

Para las demás posiciones se calcula qué columna corresponde dentro del tablero anterior. Las columnas que se encuentran después de la nueva columna deben desplazarse una posición para dejar espacio a las nuevas fichas.

Finalmente, en ambas funciones se libera la memoria utilizada anteriormente y se actualiza el puntero `tablero` junto con la cantidad de columnas. De esta forma, el tablero puede aumentar o disminuir su tamaño horizontal manteniendo la información de las fichas que no fueron eliminadas. “ “`“latex id=”7m2qpa”`

### 3.8 Procesamiento de cascadas y puntuación

Para completar el comportamiento de las combinaciones se implementó la función `procesarCascadas()`, encargada de repetir el proceso de búsqueda, eliminación, caída y rellenado mientras sigan apareciendo nuevas combinaciones.

La función comienza buscando las combinaciones existentes en el tablero mediante `buscarCombinaciones()`. Si no se encuentra ninguna, el proceso termina. En caso contrario, se actualizan las estadísticas correspondientes y se procede a eliminar las fichas encontradas.

Después de cada eliminación se ejecutan `eliminarFichas()`, `hacerCaerTablero()` y `rellenarTablero()`. Esto puede generar nuevas combinaciones como consecuencia de la caída de las fichas y de las nuevas fichas generadas. Por esta razón, el proceso se encuentra dentro de un ciclo `while`, que continúa hasta que una búsqueda no encuentra ninguna combinación.

```
while(true){

    cantidadEliminadas =
        buscarCombinaciones(tablero, eliminadas,
                            filas, columnas,
                            combinacionesDetectadas);

    if(cantidadEliminadas <= 0){
```

```

        break;
    }

    eliminarFichas(tablero, eliminadas, cantidadEliminadas);

    hacerCaerTablero(tablero, filas, columnas);

    rellenarTablero(tablero, filas, columnas);
}

```

También se lleva un registro de la cantidad de fichas eliminadas y de las combinaciones detectadas. Para la puntuación se asignan 10 puntos por cada ficha eliminada, por lo que el puntaje aumenta según la cantidad de fichas eliminadas durante el proceso.

Además, se implementó un contador de cascadas mediante `cascadasActuales`. La primera eliminación no se considera una cascada, mientras que cada nueva eliminación producida después de reorganizar el tablero incrementa este contador.

De esta manera, una jugada puede producir una o varias rondas de eliminaciones consecutivas. El proceso termina únicamente cuando, después de eliminar, hacer caer y rellenar el tablero, ya no se encuentra ninguna combinación nueva. “ “`“latex id=“q4w8ne”`”

### 3.9 Integración del programa principal

Finalmente, todas las funciones desarrolladas se integraron en el `main()`, que se encarga de controlar el flujo general del juego. Al iniciar el programa se inicializa la generación de números aleatorios y se solicitan las dimensiones iniciales del tablero.

Una vez obtenidas las dimensiones, se calcula la cantidad de bytes necesaria para representar el tablero y se reserva dinámicamente la memoria tanto para el tablero como para el arreglo utilizado para almacenar las posiciones de las fichas que serán eliminadas. Después se inicializa la memoria, se llena el tablero con fichas aleatorias y se muestra su contenido.

A partir de este punto, el programa funciona mediante un menú de opciones. Cada opción representa una acción diferente sobre el tablero. Cuando el usuario selecciona una opción, se ejecuta la función correspondiente y, después de realizar la acción, se procesan las posibles combinaciones y cascadas que se hayan generado.

En el caso de eliminar una ficha, se registra la eliminación realizada por el usuario y se ejecuta `procesarEliminacionFicha()`. Para las opciones de agregar o eliminar filas y columnas se utilizan las funciones correspondientes de modificación del tablero. Después de cualquiera de estas acciones se ajusta el tamaño del arreglo `eliminadas` mediante `redimensionarEliminadas()`, ya que el número de posiciones posibles del tablero puede haber cambiado.

Posteriormente se llama a `procesarCascadas()`, que se encarga de buscar combinaciones y repetir el proceso de eliminación, caída y rellenado mientras sigan apareciendo nuevas combinaciones. Al finalizar cada acción se muestran las estadísticas acumuladas del juego, como la cantidad de eliminaciones realizadas por el usuario, las fichas eliminadas, las combinaciones detectadas, las cascadas generadas y la puntuación obtenida.

El programa también permite consultar directamente la representación en bits del tablero mediante la opción correspondiente. Esto permite comprobar que los valores mostrados visualmente coinciden con la información que realmente se encuentra almacenada en la memoria.

El ciclo principal continúa mostrando el tablero y solicitando nuevas acciones hasta que el usuario decide finalizar el juego. Finalmente, se libera la memoria dinámica utilizada tanto por el tablero como por el arreglo de posiciones eliminadas.

De esta manera, el `main()` funciona principalmente como el punto de integración del programa, mientras que las diferentes tareas del juego se encuentran distribuidas entre las funciones desarrolladas para cada responsabilidad. “